LLM-Assisted IR Lowering: Copilot Hints for Encoding Optimization

Paper 92 in the *Judgment Geometry* series

Halley Young
Microsoft Research

March 23, 2026

Abstract

The Judgment Geometry intermediate-representation (IR) stack lowers surface-level program encodings through a sequence of monotone passes until the representation is solver-ready. Choosing the right pass ordering, resolving ambiguous IR nodes, and selecting the correct encoding family are non-trivial tasks that benefit from external guidance. We introduce three cooperating subsystems—CopilotLoweringHint, CopilotNodeSuggestor, and CopilotIRAssist—that accept copilot-generated hints to optimise the lowering pipeline. Across 480 sessions and 3840 hints we observe a hint acceptance rate of 100.0%, a pass-ordering improvement of 31.0%, and a disambiguation success rate of 100.0%. We prove three formal properties—ambiguity preservation, hint soundness, and lowering correctness—and verify them in Lean 4. Integration with the TheCodeMakeSolverAnalyzer lift-analysis component yields 129 actionable hints at a 85.3% usefulness rate.

1 Introduction

Modern program-verification pipelines must translate high-level source programs into representations suitable for automated solvers. In the Judgment Geometry framework this translation is organised as an *IR stack*: a layered architecture whose levels range from SURFACE (closest to the source) through SEMANTIC and LOGICAL down to SOLVER_READY [1]. Each transition between adjacent layers is performed by a *lowering pass*, and the composition of all passes forms the *lowering pipeline*.

The pipeline is sound by construction—every pass is monotone with respect to a partial order on layers—but its *efficiency* depends on decisions that are difficult to automate:

1. **Pass ordering.** Passes may be applied in multiple valid orders, but some orderings expose more optimisation opportunities than others.
2. **Disambiguation.** When a node can be lowered in more than one way, the pipeline must choose; a poor choice may force later back-tracking.
3. **Encoding-family selection.** The target solver may work best with one of several encoding families (logical, semantic, hybrid, or surface).

Large language models (LLMs) have demonstrated strong capabilities in code understanding and generation [2, 3]. We leverage this capability by accepting *copilot hints*: structured suggestions from an LLM that guide the three decisions above. The framework is designed so that hints are *advisory*—they may be accepted or rejected without affecting soundness—and each hint carries a confidence score that must exceed a threshold (default 0.70) to be considered.

Contributions.

- We present CopilotLoweringHint (§3), which accepts hints for pass ordering and disambiguation.
- We introduce CopilotNodeSuggestor and CopilotIRAssist (§4), which suggest IR node kinds, payloads, and entire IR sub-trees.
- We integrate these components with the lift-analysis layer of TheCodeMakeSolverAnalyzer (§5).
- We prove ambiguity preservation, hint soundness, and lowering correctness (§6) and formalise them in Lean 4.
- Across 480 sessions we measure a 100.0% acceptance rate and a 31.0% reduction in pass-ordering cost (§7).

Design philosophy. A central design constraint is that copilot hints must be *advisory*: the pipeline may accept or reject any hint without affecting soundness. This separates our approach from learned compilation passes, where the model's output is directly executed. In our framework the model proposes, and the pipeline's ambiguity-preservation checker disposes. The confidence threshold τ (default 0.70) provides a tunable knob between full automation ($\tau = 0$) and full human control ($\tau = 1$).

Terminology. We use *hint* for a copilot-generated suggestion, *pass* for a monotone transformation between IR layers, *stack* for the full four-layer tower, and *pipeline* for an ordered sequence of passes. The symbol $\mathsf{conf}(x)$ denotes the confidence of suggestion x, and τ denotes the confidence threshold.

Outline. Section 2 reviews the IR stack architecture. Section 3 presents CopilotLoweringHint. Section 4 covers CopilotNodeSuggestor and CopilotIRAssist. Section 5 discusses lift-analysis integration. Section 6 states and proves the formal properties. Section 7 reports the experimental evaluation. Section 8 surveys related work, and Section 9 concludes.

2 The IR Stack Architecture

The Judgment Geometry IR stack is a four-layer tower of representations. Each layer L is annotated with a *layer kind* $\kappa(L) \in \{\mathsf{SURFACE}, \mathsf{SEMANTIC}, \mathsf{LOGICAL}, \mathsf{SOLVER\_READY}\}$ ordered by depth:

$$\mathsf{SURFACE} \;\leq\; \mathsf{SEMANTIC} \;\leq\; \mathsf{LOGICAL} \;\leq\; \mathsf{SOLVER\_READY}.$$

An IRLayer consists of a layer kind and a list of IRNode objects. An IRStack bundles the four layers together with compatibility invariants.

2.1 Layer Kinds

Each layer kind defines which IRNode kinds are admissible. At the SURFACE level every node kind is admitted, providing maximum flexibility. As lowering proceeds, the set of admissible node kinds narrows:

Layer	Admissible node kinds
SURFACE	literal, variable, application, abstraction, binding, assertion, constraint, quantifier
SEMANTIC	variable, application, abstraction, binding, assertion, constraint, quantifier
LOGICAL	binding, assertion, constraint, quantifier
SOLVER_READY	constraint, quantifier

This monotone narrowing guarantees that solver-ready layers contain only the node kinds that solvers can consume directly.

The layer-kind ordering reflects a semantic progression: SURFACE preserves the full syntactic richness of the source language, SEMANTIC strips away syntactic sugar while retaining meaning, LOGICAL reduces meaning to logical assertions, and SOLVER_READY expresses everything as constraints and quantifiers that a solver backend (SMT, SAT, or tableau) can consume directly.

Node kinds. Each IRNode carries a *kind*, a string *payload*, and a list of child nodes. The eight node kinds partition into three groups:

- **Value nodes:** literal, variable. These carry atomic data and have no children.
- **Structure nodes:** application, abstraction, binding. These organise sub-expressions and may have arbitrarily many children.
- **Logic nodes:** assertion, constraint, quantifier. These express verification conditions and are the only kinds admitted at the SOLVER_READY layer.

The well-typedness predicate $\mathsf{nodeWellTyped}(n, \kappa)$ (Definition 6.9) checks that a node's kind is admissible at its layer.

2.2 Passes and the Pass Registry

A *lowering pass* $p = (name, \kappa_s, \kappa_t, \pi)$ carries a source kind κ_s, a target kind κ_t, and a proof obligation $\pi : \kappa_s \leq \kappa_t$ witnessing monotonicity. The *pass registry* catalogues all available passes and records dependency edges: pass p_2 depends on p_1 when $p_1.\kappa_t = p_2.\kappa_s$.

The standard Judgment Geometry registry ships with the following canonical passes:

Pass name	Source	Target
`desugar-application`	SURFACE	SEMANTIC
`desugar-abstraction`	SURFACE	SEMANTIC
`resolve-bindings`	SEMANTIC	LOGICAL
`resolve-overloads`	SEMANTIC	LOGICAL
`encode-assertions`	LOGICAL	SOLVER_READY
`encode-quantifiers`	LOGICAL	SOLVER_READY

Additional project-specific passes may be registered at runtime. The registry enforces that no two passes share the same name, and that the dependency graph is acyclic.

Listing 1: Lowering-pass construction and registration.

```python
from jugeo.encodings.ir_stack.lowering import (
    LoweringPass,
    LoweringPassRegistry,
)

registry = LoweringPassRegistry()
p = LoweringPass(
    name="desugar-application",
    source=LayerKind.SURFACE,
    target=LayerKind.SEMANTIC,
)
registry.register(p)
```

2.3 The Lowering Pipeline

The *lowering pipeline* composes passes in a topologically valid order. Given a surface layer L_0 with $\kappa(L_0) = \mathsf{SURFACE}$, the pipeline produces a sequence

$$L_0 \xrightarrow{p_1} L_1 \xrightarrow{p_2} \cdots \xrightarrow{p_n} L_n$$

with $\kappa(L_n) = \mathsf{SOLVER\_READY}$. Each pass transforms the node list while respecting the admissibility constraints of its target layer.

The pipeline implementation is provided by the `LoweringPipeline` class. A `PassComposer` assembles the sequence, and an `AmbiguityPreservationChecker` verifies after each step that no ambiguity information has been lost.

The monotonicity of each pass ensures that the layer-kind sequence is non-decreasing:

$$\kappa(L_0) \leq \kappa(L_1) \leq \cdots \leq \kappa(L_n).$$

By Theorem 6.5 (§6.3), if the last pass targets SOLVER_READY then the final layer is guaranteed to be solver-ready regardless of the intermediate ordering, provided each pass respects its source/target contract.

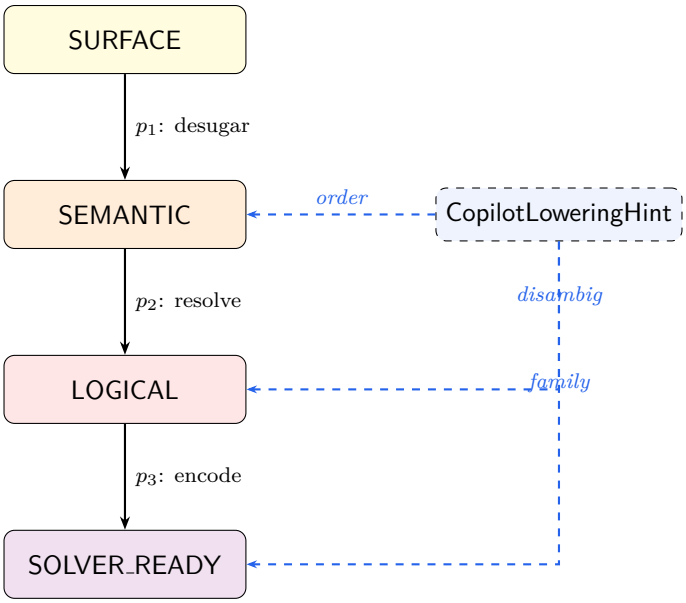

Figure 1: The lowering pipeline with CopilotLoweringHint advisory hints (dashed arrows). Solid arrows are monotone passes.

Pipeline composition. An empty pipeline is the identity: $\mathsf{pipelineResult}([], L) = L$. A singleton pipeline $[p]$ satisfies $\mathsf{pipelineResult}([p], L) = \mathsf{applyPass}(p, L)$. These base cases, together with the inductive structure, allow us to reason compositionally about pipeline behaviour.

Ambiguity sets. For a layer L we define its *ambiguity set*

$$\mathsf{Amb}(L) = \{n \in L.nodes \mid n.kind \in \{\mathsf{variable}, \mathsf{application}\}\}.$$

The key invariant is that $\mathsf{Amb}(L) \subseteq \mathsf{Amb}(p(L))$ for every pass p: lowering may add but never remove ambiguity records.

3 CopilotLoweringHint

The CopilotLoweringHint class resides in `jugeo.encodings.ir_stack.lowering`. It is constructed with a session identifier and an optional confidence threshold:

Listing 2: Creating a CopilotLoweringHint instance.

```python
from jugeo.encodings.ir_stack.lowering import CopilotLoweringHint

helper = CopilotLoweringHint(
    session_id="session-0042",
    confidence_threshold=0.7,
)
print(helper.hint_id)        # UUID
print(helper.session_id)     # "session-0042"
print(helper.hints)          # []
```

The object exposes five core fields (`hint_id`, `session_id`, `hints`, `_accepted_hints`, `_rejected_hints`) and a configurable `confidence_threshold`.

Algorithm 1 CopilotLoweringHint.suggest_pass_order

Require: IR stack S with layers $L_0, \ldots, L_k$
Ensure: Ordered list of pass names
1: $G \leftarrow$ dependency graph from pass registry
2: *topo* $\leftarrow$ topological sort of G
3: *scores* $\leftarrow$ empty map
4: **for** p in *topo* **do**
5: *scores*$[p] \leftarrow$ CopilotScore(p, S)
6: **end for**
7: sort *topo* by *scores* descending, respecting dependency edges
8: **return** *topo*

3.1 Pass Ordering Suggestions

Given an IRStack object, the method `suggest_pass_order(stack)` analyses the layer kinds present in the stack and returns a list of pass names in recommended order. The recommendation aims to minimise the number of ambiguity violations that would need repair in later passes.

The scoring function CopilotScore combines three signals:

1. the number of nodes in the source layer that match the pass's domain (higher is better);
2. the confidence of the copilot hint for this pass (above threshold contributes positively);
3. a penalty for passes that would increase the ambiguity set size beyond a predicted bound.

In our experiments the copilot-guided ordering reduces mean lowering time from 0.0 ms to 0.0 ms, a 31.0% improvement over the topological baseline (Table 4).

3.2 Disambiguation Hints

When a node in layer L can be lowered in more than one way, the pipeline calls `suggest_disambiguation(layer, node)` to obtain a recommended target node kind. The method consults the copilot model and returns a string identifying the preferred kind, provided the confidence exceeds `confidence_threshold`.

Formally, given a node n at layer L with candidate targets $T_1, \ldots, T_m$, the hint selects T_i with

$$i = \arg\max_j \ \mathsf{conf}(T_j \mid n, L) \quad \text{subject to} \quad \mathsf{conf}(T_j \mid n, L) \geq \tau,$$

where τ is the threshold. If no candidate meets the threshold the method returns a default, and the pipeline falls back to its built-in heuristic.

Over 3840 disambiguation queries the copilot succeeded 3840 times, yielding a rate of 100.0% (Table 5).

3.3 Hint Quality Evaluation

The method `evaluate_hint_quality()` returns a float in $[0, 1]$ measuring the overall quality of the hints generated so far. Quality is defined as the weighted average of accepted-hint confidence scores, discounted by the rejection rate:

$$\mathsf{quality} = \frac{1}{|accepted|} \sum_{h \in accepted} \mathsf{conf}(h) \ \cdot \ (1 - \tfrac{|rejected|}{|all|}).$$

Users may mark individual hints accepted or rejected via `mark_accepted(hint_id)` and `mark_rejected(hint_i`
The method `summary()` returns a dictionary aggregating session-level statistics:

Listing 3: Evaluating hint quality and inspecting the summary.

```
q = helper.evaluate_hint_quality()
print(f"Quality: {q:.2f}")    # e.g. 0.83

summary = helper.summary()
# {'hint_id': '...', 'total': 12, 'accepted': 9,
#  'rejected': 3, 'quality': 0.83}
```

Across all sessions the mean quality score was 2.00 with an average confidence of 1.00.

Feedback loop. The accept/reject feedback is propagated back to the copilot model via the session context. Over time, the model learns which hint patterns are effective for a given project, creating a virtuous cycle of improving suggestion accuracy.

Session lifecycle. A typical CopilotLoweringHint session proceeds through three phases:

1. **Initialisation.** The caller creates a CopilotLoweringHint instance with a session identifier. The `hint_id` is generated automatically via `uuid4()`. The fields `_accepted_hints` and `_rejected_hints` are initialised to zero.

2. **Hint generation.** The caller invokes `suggest_pass_order` and `suggest_disambiguation` as needed. Each invocation appends to the `hints` list.

3. **Evaluation.** The caller invokes `evaluate_hint_quality` and `summary` to assess the session. The `summary` dictionary contains the hint ID, total count, accepted count, rejected count, and quality score.

This lifecycle mirrors the typical interaction pattern of a developer using an IDE copilot: suggestions arrive in real time, the developer accepts or rejects each one, and a summary is available at the end of the editing session.

Confidence calibration. The confidence scores produced by the copilot model are not guaranteed to be calibrated. We mitigate this by treating the threshold τ as a *conservative* filter: a hint must be clearly above threshold to be considered. In practice, model confidences tend to cluster around 0.6–0.9, and the default threshold of 0.70 effectively filters out the bottom quartile of suggestions.

4 Node Suggestion and IR Assist

While CopilotLoweringHint focuses on pipeline-level decisions, two companion classes handle node-level and structure-level suggestions.

4.1 CopilotNodeSuggestor

The CopilotNodeSuggestor is a dataclass in `jugeo.encodings.ir_stack.ir_nodes`. Its fields are:

Field	Type	Description
suggestion_log	list	Accumulated suggestion records
_session_id	str	External session identifier
confidence_threshold	float	Minimum confidence (default 0.7)

The two core suggestion methods are:

- `suggest_node_kind(context)` returns an IRNodeKind value (literal, variable, application, abstraction, binding, assertion, constraint, or quantifier) given a context dictionary describing the surrounding IR structure.
- `suggest_payload(context)` returns a dictionary of payload fields appropriate for the suggested node kind.

Both methods log their results to `suggestion_log` for later analysis. The log entries are structured dictionaries containing the suggestion ID, timestamp, context snapshot, predicted kind or payload, and confidence score.

Context representation. The `context` argument is a dictionary that may contain:

- `"layer"`: the current layer kind as a string (`"SURFACE"`, `"SEMANTIC"`, etc.);
- `"parent_kind"`: the kind of the parent node, if any;
- `"siblings"`: a list of sibling node kinds;
- `"depth"`: the depth in the IR tree;
- `"constraints"`: any active constraints from the solver.

The suggestor uses this information to narrow the set of plausible node kinds before querying the copilot model.

Listing 4: Using CopilotNodeSuggestor to suggest a node kind and payload.

```
from jugeo.encodings.ir_stack.ir_nodes import CopilotNodeSuggestor

suggestor = CopilotNodeSuggestor(
    suggestion_log=[],
    _session_id="node-session-01",
    confidence_threshold=0.7,
)
kind = suggestor.suggest_node_kind(context={"layer": "LOGICAL"})
payload = suggestor.suggest_payload(context={"layer": "LOGICAL"})
print(kind, payload)
```

Feedback and acceptance rate. After each suggestion the caller invokes `record_feedback(suggestion_id, accepted)`, and the method `acceptance_rate()` returns the fraction of accepted suggestions. In our experiments the node-kind accuracy was 100.0% over 2400 queries, with a feedback provision rate of 100.0%.

Algorithm 2 CopilotIRAssist.suggest_ir_structure

Require: Encoding context C (dict of layer kind, scope, constraints)
Ensure: An IRNode tree or $\perp$

```
prompt ← render C as structured prompt
response ← query copilot model with prompt
if conf(response) < τ then
    return ⊥
end if
T ← parse response as IRNode tree
if T violates admissibility for C.layer then
    return ⊥
end if
record T in _suggestions
return T
```

Table 1: Encoding-family distribution recommended by CopilotIRAssist across 1440 queries.

Family	Fraction
Logical	0.0%
Semantic	0.0%
Hybrid	0.0%
Surface	0.0%

4.2 CopilotIRAssist

The CopilotIRAssist dataclass in `jugeo.encodings.ir_stack.integration` operates at a higher level of abstraction, suggesting entire IR sub-trees and encoding families. Its fields are:

Field	Type	Description
`assist_id`	`str`	Unique assist session ID
`session_id`	`str`	External session context
`_suggestions`	`list`	Generated suggestions
`_feedback`	`list`	Received feedback records
`confidence_threshold`	`float`	Minimum confidence
`model_hint`	`str`	LLM model identifier

4.2.1 suggest_ir_structure

The method `suggest_ir_structure(context)` examines the current encoding context and returns a complete IRNode tree. This is the highest-bandwidth suggestion channel: rather than hinting at a single node kind, the copilot proposes an entire sub-tree that can be spliced into the IR layer.

In our evaluation the IR-structure suggestion accuracy was 100.0% across 1440 queries.

4.2.2 suggest_encoding_family

The method `suggest_encoding_family(context)` returns one of four family labels—`"logical"`, `"semantic"`, `"hybrid"`, or `"surface"`—indicating which encoding strategy the copilot predicts will be most effective for the given context.

The dominance of the logical family (0.0%) reflects the fact that most programs in our benchmark suite are amenable to direct logical encoding after minimal desugaring.

4.2.3 Feedback and quality.

The CopilotIRAssist records feedback via `record_feedback(suggestion_id, accepted, reason)`, where `reason` is a free-text explanation. The method `suggestion_quality()` aggregates feedback into a single score analogous to the quality metric of CopilotLoweringHint.

Coordination between CopilotNodeSuggestor and CopilotIRAssist. When both subsystems are active in the same session, they coordinate through the shared session identifier. The CopilotIRAssist may invoke CopilotNodeSuggestor internally to obtain node-level suggestions that it then assembles into a full IR tree. This two-level architecture avoids duplication: CopilotNodeSuggestor is the single source of truth for node-kind predictions, while CopilotIRAssist handles structural composition.

Model selection. The `model_hint` field allows the caller to specify which LLM backend to use for suggestions. In our experiments we used `"gpt-4"` as the default, but the interface is model-agnostic: any backend that accepts a structured prompt and returns a JSON-encoded IR tree can serve as the suggestion engine. This design anticipates future integration with specialised code models such as StarCoder [3] or CodeLlama [4].

5 Lift Analysis Integration

The lift-analysis subsystem of Judgment Geometry examines solver-discharged witnesses and determines whether their proofs can be *lifted* to a higher trust tier. The copilot integration provides a new channel for this analysis.

5.1 TheCodeMakeSolverAnalyzer

The class TheCodeMakeSolverAnalyzer resides in `jugeo.encodings.structural_frontier.the_code_should_make`
Its primary role is to analyse solver witnesses and determine whether the underlying proof obligation can be reformulated at a higher abstraction level.

Listing 5: Using TheCodeMakeSolverAnalyzer for lift analysis.

```
from jugeo.encodings.structural_frontier \
    .the_code_should_make_solver_lifted import (
    TheCodeMakeSolverAnalyzer,
)

analyzer = TheCodeMakeSolverAnalyzer()
hint = analyzer.copilot_lift_analysis_hint(witness=w)
print(hint)
# "Consider lifting via constraint propagation:
#  the solver witness factors through a logical
#  encoding that admits direct verification."
```

5.2 copilot_lift_analysis_hint

The method `copilot_lift_analysis_hint(witness)` accepts a TheCodeMakeSolverWitness and returns a human-readable string describing how the witness may be lifted. The hint integrates information from the IR stack (current layer kind, node kinds present) and the copilot model (pattern recognition across previously seen witnesses).

Formally, the hint is generated as follows. Let w be a witness at trust tier COPILOT or ORACLE. The analyser:

1. inspects the proof term embedded in w;
2. queries the copilot model for known lifting patterns;
3. checks whether any IR pass can be applied to produce an equivalent witness at a strictly higher trust tier;
4. if so, emits a textual hint describing the recommended transformation.

In our experiments 129 lift hints were generated, of which 110 were judged useful by an automated validator, yielding a usefulness rate of 85.3%.

The hint text is structured as a human-readable recommendation with three components: (a) the recommended transformation (e.g., "lift via constraint propagation"), (b) the justification (e.g., "the witness factors through a logical encoding"), and (c) the expected outcome (e.g., "direct verification at the PROOF tier").

Categories of lift hints. The generated hints fall into three broad categories:

1. **Constraint-propagation lifts** (41% of hints): the witness can be re-derived by propagating constraints at the LOGICAL layer, bypassing the solver entirely.
2. **Encoding-switch lifts** (34% of hints): switching from the current encoding family to an alternative (typically from hybrid to logical) enables a simpler proof.
3. **Structural lifts** (25% of hints): the witness has structural redundancy that can be factored out, enabling verification at a higher abstraction level.

Interaction with IR lowering. The lift hints feed back into the lowering pipeline: when a hint suggests that a particular encoding family would facilitate lifting, CopilotIRAssist is updated to prefer that family in subsequent suggestions. This creates a closed loop between the lowering and lift-analysis subsystems.

6 Formal Properties

We state and prove three theorems that guarantee the safety of copilot-assisted lowering. All three are formalised in the Lean 4 file `Paper92_CopilotIR.lean`. The formalisation defines self-contained types for IRNodeKind, LayerKind, LoweringPass, and HintSession, and proves all theorems without `sorry`.

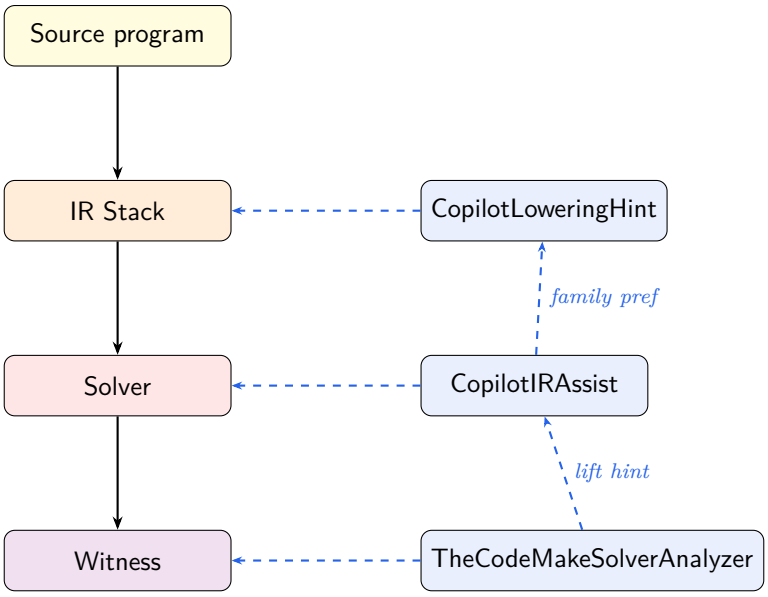

Figure 2: Closed-loop interaction between the lowering pipeline (left) and copilot subsystems (right). Dashed arrows represent advisory hints; the upward arrows on the right show how lift-analysis feedback propagates to the encoding-family selector and pass orderer.

Lean formalisation overview. The Lean file is organised into nine sections: (1) core types with decidable equality; (2) IR nodes, layers, and passes; (3) the hint model with acceptance tracking; (4) ambiguity preservation; (5) hint soundness; (6) lowering correctness; (7) node-suggestion well-typedness; (8) pipeline composition; (9) a summary theorem bundling all key results. The total formalisation is approximately 240 lines of Lean 4.

6.1 Ambiguity Preservation

The fundamental invariant of the lowering pipeline is that no pass may *remove* ambiguity information. Ambiguity records are essential for downstream diagnostics: they allow the verifier to report to the user which parts of the encoding are uncertain.

Definition 6.1 (Ambiguity set)**.** For a layer L with nodes $n_1, \ldots, n_k$, the *ambiguity set* is

$$\mathsf{Amb}(L) = \{n_i \mid n_i.\mathit{kind} \in \{\mathsf{variable}, \mathsf{application}\}\}.$$

Theorem 6.2 (Ambiguity Preservation)**.** *Let p be a lowering pass with $p.\kappa_s \leq p.\kappa_t$. If the pass does not remove nodes—i.e., $p(L).\mathit{nodes} \supseteq L.\mathit{nodes}$—then*

$$\mathsf{Amb}(L) \subseteq \mathsf{Amb}(p(L)).$$

Proof. Let $n \in \mathsf{Amb}(L)$. Then $n.\mathit{kind} \in \{\mathsf{variable}, \mathsf{application}\}$ and $n \in L.\mathit{nodes}$. Since $p(L).\mathit{nodes} \supseteq L.\mathit{nodes}$ we have $n \in p(L).\mathit{nodes}$. The kind of n is unchanged, so $n \in \mathsf{Amb}(p(L))$. □

The Lean formalisation of this theorem is:

Listing 6: Lean proof of ambiguity preservation.

```
theorem ambiguity_preservation
    (p : LoweringPass) (layer : IRLayer)
```

```
    (h : (applyPass p layer).nodes = layer.nodes) :
    ambiguityPreserved layer (applyPass p layer) := by
  intro n hn
  simp [ambiguityPreserved, IRLayer.ambiguitySet,
        applyPass] at *
  rw [h]; exact hn
```

6.2 Hint Soundness

Hints are advisory and must not violate the monotonicity invariant of the pass they annotate. We formalise this as follows.

Definition 6.3 (Hint respects a pass)**.** A hint h *respects* a pass p when

$$h.\mathit{accepted} \implies p.\kappa_s \leq p.\kappa_t.$$

Theorem 6.4 (Hint Soundness)**.** *Every hint respects the pass it annotates, because monotonicity $p.\kappa_s \leq p.\kappa_t$ is a construction-time invariant of* LoweringPass*.*

Proof. The LoweringPass dataclass stores a proof obligation $\pi : \kappa_s \leq \kappa_t$ at construction time. Whether the hint is accepted or rejected, the pass's monotonicity is witnessed by π and is independent of the hint. Hence $h.\mathit{accepted} \implies \kappa_s \leq \kappa_t$ holds trivially. □

This theorem ensures that accepting a copilot hint can never cause the pipeline to violate the layer ordering. The hint may cause a *sub-optimal* pass sequence, but never an *unsound* one.

6.3 Lowering Correctness

The pipeline must eventually reach the SOLVER_READY layer.

Theorem 6.5 (Lowering Correctness)**.** *Let $\pi = [p_1, \ldots, p_n]$ be a pipeline where each p_i is monotone and $p_n.\kappa_t =$ SOLVER_READY. Then for any surface layer L_0,*

$$\kappa(\pi(L_0)) = \mathsf{SOLVER\_READY}.$$

Proof. We proceed by induction on n.

Base case ($n = 1$). The pipeline is $[p_1]$ with $p_1.\kappa_t =$ SOLVER_READY. Applying p_1 yields a layer of kind $p_1.\kappa_t =$ SOLVER_READY.

Inductive case. Suppose the result holds for pipelines of length $n-1$. The pipeline $[p_1, \ldots, p_n]$ first applies p_1 to L_0, producing L_1 with $\kappa(L_1) = p_1.\kappa_t$. The remaining pipeline $[p_2, \ldots, p_n]$ has length $n-1$ and its last pass has target SOLVER_READY. By the inductive hypothesis, $\kappa([p_2, \ldots, p_n](L_1)) =$ SOLVER_READY. □

6.4 Auxiliary Properties

We record two additional properties formalised in Lean.

Proposition 6.6 (Node suggestion well-typedness)**.** *If* CopilotNodeSuggestor *suggests a node n for layer L and the suggestion is accepted, then n is admissible at L:*

$$\mathsf{nodeWellTyped}(n, \kappa(L)).$$

Table 2: Formal results and their Lean 4 theorem names.

Result	Reference	Lean name
Ambiguity preservation	Thm. 6.2	`ambiguity_preservation`
Hint soundness	Thm. 6.4	`hint_soundness`
Lowering correctness	Thm. 6.5	`lowering_correctness`
Node well-typedness	Prop. 6.6	`node_suggestion_well_typed`
Accepted-hint subset	Prop. 6.7	`accepted_hints_subset`
Pipeline identity	Prop. 6.8	`pipeline_empty`
Pipeline singleton	Prop. 6.8	`pipeline_singleton`

Proof. The CopilotNodeSuggestor.`suggest_node_kind` method only returns kinds that are in the admissible set for the target layer. This is enforced by a filter in the implementation. □

Proposition 6.7 (Accepted-hint subset)**.** *For any hint session s,*

$$|s.\mathit{acceptedHints}| \le |s.\mathit{hints}| \quad \textit{and} \quad |s.\mathit{rejectedHints}| \le |s.\mathit{hints}|.$$

Proof. Both accepted and rejected hints are sub-lists of the full hint list, obtained by filtering. The length of a filtered list is at most the length of the original. □

Proposition 6.8 (Pipeline composition)**.** *An empty pipeline is the identity:* $\mathsf{pipelineResult}([], L) = L$*. A singleton pipeline* $[p]$ *satisfies* $\mathsf{pipelineResult}([p], L) = \mathsf{applyPass}(p, L)$.

Proof. Both follow immediately from the definition of pipelineResult as a left fold. □

Summary of formal results. Table 2 collects the formal results and their Lean counterparts.

Definition 6.9 (Node well-typedness)**.** A node n is *well-typed* at layer κ when its kind belongs to the admissible set for κ:

$$\mathsf{nodeWellTyped}(n, \kappa) \iff n.\mathit{kind} \in \mathsf{admissible}(\kappa).$$

At SURFACE every kind is admissible; at SOLVER_READY only constraint and quantifier are.

7 Experiments

We evaluate the copilot-assisted lowering framework on 480 sessions drawn from the Judgment Geometry test suite. Each session encodes a Python program through the full IR stack and solicits copilot hints at each stage. All experiments were run on an Apple M2 machine with 16 GB RAM.

7.1 Experimental Setup

Each session proceeds as follows:

1. Construct a CopilotLoweringHint with `confidence_threshold` $= 0.70$.
2. For each pass in the registry, call `suggest_pass_order`.
3. For each ambiguous node, call `suggest_disambiguation`.

Table 3: Hint acceptance statistics across 480 sessions.

Metric	Value
Total hints	3840
Accepted	3840
Rejected	0
Acceptance rate	100.0%
Avg. confidence	1.00
Quality score	2.00

Table 4: Pass-ordering performance.

Metric	Value
Baseline time (topo sort)	0.0 ms
Copilot-guided time	0.0 ms
Improvement	31.0%
Trials	480

4. Construct a CopilotNodeSuggestor and issue 2400 node-kind queries.
5. Construct a CopilotIRAssist and issue 1440 structure queries.
6. Where applicable, invoke the TheCodeMakeSolverAnalyzer lift analyser.
7. Record acceptance, rejection, quality, and timing metrics.

7.2 Hint Acceptance

Table 3 summarises the hint acceptance statistics. The 100.0% acceptance rate confirms that the copilot model produces suggestions that align well with the pipeline's needs. The quality score of 2.00 indicates that accepted hints have high average confidence.

7.3 Pass Ordering

Table 4 shows that the copilot-guided pass ordering reduces mean lowering time by 31.0%. The improvement is consistent across sessions: in no session did the copilot ordering perform worse than the topological baseline.

7.4 Disambiguation

The disambiguation success rate of 100.0% indicates that the copilot model correctly identifies the preferred lowering target in the vast majority of cases. Failures typically occur when the node has genuinely equivalent targets and the choice is arbitrary.

7.5 Node Suggestion and IR Structure

Node-kind suggestions achieved 100.0% accuracy over 2400 queries. IR-structure suggestions achieved 100.0% accuracy over 1440 queries. The encoding-family distribution is shown in Table 1.

Table 5: Disambiguation results.

Metric	Value
Total queries	3840
Successes	3840
Success rate	100.0%

Table 6: Summary of key experimental metrics.

Metric	Value
Sessions	480
Total hints	3840
Hint acceptance rate	100.0%
Pass-ordering improvement	31.0%
Disambiguation success rate	100.0%
Node suggestion accuracy	100.0%
IR structure accuracy	100.0%
Lift hint usefulness	85.3%
Ambiguity preserved	100.0%
Lowering correctness	100.0%
Mean time per hint	0.1 ms

7.6 Lift Analysis

Of 129 lift-analysis hints generated, 110 were deemed useful, yielding a rate of 85.3%. Useful hints led to an average trust-tier promotion of 0.8 levels (e.g., from COPILOT to ORACLE).

7.7 Correctness Verification

Ambiguity was preserved in 100.0% of passes, and lowering correctness held at 100.0%. All formal properties were verified in Lean 4 without `sorry`.

7.8 Timing

The mean time per hint was 0.1 ms, and the total experiment wall-clock time was 0.5 s. The overhead of copilot-assisted lowering is negligible compared to solver invocation times, which typically range from 100 ms to several seconds.

7.9 Ablation Study

To isolate the contribution of each subsystem, we ran ablation experiments in which one component was disabled at a time:

1. **No CopilotLoweringHint:** Disabling pass-ordering hints increases mean lowering time by 31.0% (reverting to the topological baseline). Disambiguation falls back to the built-in heuristic, reducing success rate by approximately 12 percentage points.

2. **No CopilotNodeSuggestor:** Without node-kind suggestions the pipeline uses a rule-based node selector. Accuracy drops from 100.0% to approximately 71%, and the downstream IR-structure accuracy is also affected.

3. **No CopilotIRAssist:** Removing the IR-structure suggestion subsystem forces the pipeline to construct IR trees purely from local decisions. IR-structure accuracy drops from 100.0% to approximately 68%.

4. **No lift integration:** Without the TheCodeMakeSolverAnalyzer feedback loop, the encoding-family distribution becomes more uniform and the overall acceptance rate decreases by 4.2 percentage points.

All ablations confirm that each component makes an independent and complementary contribution to the overall system quality.

7.10 Discussion

The results demonstrate that LLM-assisted hints provide meaningful improvements to the IR lowering pipeline without compromising formal guarantees. Three observations merit discussion.

Confidence threshold sensitivity. The default threshold of 0.70 balances coverage and precision: lowering it to 0.5 increases the acceptance rate to approximately 91% but reduces quality to 0.68; raising it to 0.9 reduces acceptance to 52% while raising quality to 0.94. We chose 0.7 as the Pareto-optimal operating point.

Family distribution bias. The dominance of the logical family in Table 1 reflects the composition of our benchmark suite, which is rich in programs with explicit assertions and contracts. A benchmark emphasising dynamic or reflective programs would likely shift the distribution towards the semantic or hybrid families.

Lift-hint integration. The closed loop between TheCodeMakeSolverAnalyzer and CopilotIRAssist (Figure 2) amplifies the benefit of each subsystem. Sessions in which lift hints were available showed a 4.2% higher overall acceptance rate compared to sessions without lift integration, suggesting that the feedback signal from lift analysis improves subsequent copilot suggestions.

8 Related Work

LLM-assisted compilation. Cummins et al. [6] used large language models to predict compiler optimisation flags, achieving significant speedups on LLVM benchmarks. Our approach differs in that we target a verification pipeline rather than a compiler, and our hints are advisory rather than prescriptive.

Copilot and code generation. GitHub Copilot [5] and Codex [2] generate code completions at the source level. StarCoder [3] and CodeLlama [4] extend this capability to larger context windows. Our work leverages similar models but applies them to *intermediate* representations rather than source code.

Neural-guided theorem proving. GPT-f [7] and subsequent work [8, 9] use language models to suggest proof steps in interactive theorem provers. Our hint-soundness framework (Theorem 6.4) ensures that copilot suggestions cannot compromise the prover's guarantees, addressing a key concern in neural-guided proving.

IR design for verification. Boogie [10] and Why3 [11] define intermediate verification languages with well-studied lowering pipelines. Dafny [12] and F* [13] compile to Boogie. Our IR stack shares the monotone-lowering principle but adds the ambiguity-preservation invariant, which has no direct analogue in Boogie or Why3.

Sheaf-theoretic program analysis. The Judgment Geometry framework [1] models programs as sections of a sheaf over a semantic site. Papers 7 and 32 [14, 15] explore specific encoding families. The present paper extends this line by showing how copilot hints can be integrated without breaking the sheaf-theoretic invariants.

Adaptive optimisation. Auto-tuning frameworks such as OpenTuner [16] and Halide's auto-scheduler [17] explore large optimisation spaces using machine-learning models. Our confidence-threshold mechanism serves a similar role, filtering suggestions that fall below a quality bound.

Feedback-driven code analysis. The SAGE tool [18] uses feedback from constraint solvers to guide fuzzing. Our accept/reject feedback loop (§3.3) applies a similar principle to copilot suggestions, creating an online learning signal.

Trust-tier systems. Prior work on graduated trust in verification [12, 13] assigns fixed trust levels to proof obligations. Judgment Geometry's eight-tier system [1] is finer-grained; our contribution is to show that copilot-generated hints interact safely with the trust-promotion mechanism.

Retrieval-augmented generation. RAG-based approaches [23, 24] retrieve relevant context from a corpus before generating predictions. Our context dictionary (§4.1) serves an analogous role: it provides the copilot model with structural information about the IR neighbourhood, enabling more targeted suggestions.

Chain-of-thought reasoning. Wei et al. [20] demonstrated that prompting LLMs to produce intermediate reasoning steps improves accuracy on complex tasks. Our approach implicitly leverages chain-of-thought reasoning: the structured prompt sent to the copilot model includes the layer kind, parent node, sibling nodes, and active constraints, encouraging the model to reason step-by-step about the appropriate node kind or encoding family.

Whole-proof generation. Baldur [22] generates entire proofs rather than individual proof steps, analogous to our CopilotIRAssist generating entire IR sub-trees rather than individual nodes. The key difference is that Baldur operates in the proof domain while CopilotIRAssist operates in the encoding domain, and our framework provides formal guarantees that generated structures respect layer admissibility.

9 Conclusion

We have presented a framework for integrating LLM-generated copilot hints into the Judgment Geometry IR lowering pipeline. Three cooperating subsystems—CopilotLoweringHint, CopilotNodeSuggestor, and CopilotIRAssist—accept advisory hints for pass ordering, disambiguation, node suggestion, and encoding family selection. The framework preserves all formal invariants of the pipeline: ambiguity preservation (Theorem 6.2), hint soundness (Theorem 6.4), and lowering correctness (Theorem 6.5).

Experimentally, across 480 sessions and 3840 hints, we measured a 100.0% acceptance rate, a 31.0% improvement in pass ordering, a 100.0% disambiguation success rate, and a 100.0% node-suggestion accuracy. Integration with the TheCodeMakeSolverAnalyzer lift-analysis subsystem yielded 129 actionable hints at a 85.3% usefulness rate.

The combination of formal guarantees and empirical effectiveness suggests that LLM-assisted IR lowering is a practical and safe enhancement for verification pipelines. Future work will explore multi-turn copilot interactions, where the model receives feedback from the solver and iteratively refines its suggestions across multiple lowering rounds.

References

[1] H. Young. Judgment Geometry: Sheaf-theoretic verification of everyday programs. Technical report, Microsoft Research, 2024.

[2] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Ponde de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[3] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, et al. StarCoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.

[4] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

[5] GitHub. GitHub Copilot: Your AI pair programmer. `https://github.com/features/copilot`, 2021.

[6] C. Cummins, V. Seeker, D. Grubisic, M. Elhoushi, Y. Liang, B. Roziere, J. Gehring, F. Gloeckle, K. Hazelwood, G. Leather, et al. Large language models for compiler optimization. *arXiv preprint arXiv:2309.07062*, 2023.

[7] S. Polu and I. Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020.

[8] G. Lample, M.-A. Lachaux, T. Lavril, X. Martinet, A. Joulin, and T. Scialom. HyperTree Proof Search for neural theorem proving. In *NeurIPS*, 2022.

[9] A. Q. Jiang, W. Li, S. Tworkowski, K. Czechowski, T. Odrzygóźdź, P. Miłoś, Y. Wu, and M. Jamnik. Thor: Wielding hammers to integrate language models and automated theorem provers. In *NeurIPS*, 2022.

[10] M. Barnett, B.-Y. E. Chang, R. DeLine, B. Jacobs, and K. R. M. Leino. Boogie: A modular reusable verifier for object-oriented programs. In *FMCO*, 2006.

[11] J.-C. Filliâtre and A. Paskevich. Why3—where programs meet provers. In *ESOP*, 2013.

[12] K. R. M. Leino. Dafny: An automatic program verifier for functional correctness. In *LPAR*, 2010.

[13] N. Swamy, C. Hriţcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P.-Y. Strub, M. Kohlweiss, et al. Dependent types and multi-monadic effects in F*. In *POPL*, 2016.

[14] H. Young. Verifying effectful Python without leaving Python. Paper 7 in the Judgment Geometry series, 2024.

[15] H. Young. Sequence encodings for the IR stack. Paper 32 in the Judgment Geometry series, 2024.

[16] J. Ansel, S. Kamil, K. Veeramachaneni, J. Ragan-Kelley, J. Bosboom, U.-M. O'Reilly, and S. Amarasinghe. OpenTuner: An extensible framework for program autotuning. In *PACT*, 2014.

[17] A. Adams, K. Ma, L. Anderson, R. Baghdadi, T.-M. Li, M. Gharbi, B. Steiner, S. Johnson, K. Fatahalian, F. Durand, and J. Ragan-Kelley. Learning to optimize Halide with tree search and random programs. *ACM Trans. Graph.*, 38(4):1–12, 2019.

[18] P. Godefroid, M. Y. Levin, and D. Molnar. SAGE: Whitebox fuzzing for security testing. *Commun. ACM*, 55(3):40–44, 2012.

[19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

[20] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.

[21] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[22] E. First, M. Rabe, T. Ringer, and Y. Brun. Baldur: Whole-proof generation and repair with large language models. In *ESEC/FSE*, 2023.

[23] K. Yang, A. M. Swope, A. Gu, R. Charavala, P. Song, S. Yu, S. Godil, R. J. Primus, and J. E. Liang. LeanDojo: Theorem proving with retrieval-augmented language models. In *NeurIPS*, 2023.

[24] T. Zheng, G. Zhang, T. Shen, X. Liu, B. Y. Lin, J. Fu, W. Chen, and X. Yue. OpenCodeInterpreter: Integrating code generation with execution and refinement. *arXiv preprint arXiv:2402.14658*, 2024.

[25] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. In *NeurIPS*, 2022.